Project 8

Python Network Automation

Automated Router Config Backup with SSH, Python, and Cron Scheduling

Author	Daksh Patel
Certification	CCNA Certified
Platform	Ubuntu Server 24.04 (VirtualBox)
Languages	Python 3.12, Bash
Libraries	Paramiko, Netmiko
Date	April 2026
GitHub	github.com/Daksh2601
LinkedIn	linkedin.com/in/pateldaksh

Table of Contents

1. Project Overview
2. Environment Setup
3. Mock Router Simulator
4. Backup Script
5. Cron Job Scheduling
6. Email Alerting
7. Verification & Testing
8. Project File Structure
9. Lessons Learned

1. Project Overview

This project automates the process of backing up network device configurations using Python. A Python script connects to routers via SSH, retrieves the running configuration, and saves it to timestamped backup files. The script is scheduled to run automatically via a Linux cron job, and includes email alerting for failed backups.

Objectives

Objective	Implementation
SSH into routers automatically	Paramiko library for SSH connections
Pull running configurations	Send 'show running-config' command via SSH
Save with timestamps	Files named Device_YYYY-MM-DD_HH-MM-SS.txt
Schedule automated backups	Linux cron job running daily at 2:00 AM
Alert on failures	Email notification via SMTP on backup failure

2. Environment Setup

Component	Details
Operating System	Ubuntu Server 24.04.4 LTS (aarch64)
Hostname	DNS-DHCP-SER
IP Address	192.168.56.2 (VirtualBox Host-Only)
Python Version	3.12.3
Pip Version	24.0
Key Libraries	paramiko 4.0.0, netmiko
Virtualization	VirtualBox on Mac M1 (Apple Silicon)

Installation Commands

```
$ sudo apt update
$ sudo apt install python3 python3-pip -y
$ pip3 install netmiko --break-system-packages
$ pip3 install paramiko --break-system-packages
```

3. Mock Router Simulator

Since Cisco Packet Tracer does not support external SSH connections, a mock router simulator was built using Python and the Paramiko library. This simulator creates two virtual routers (Router-HQ and Router-Branch) that listen on separate SSH ports and respond to network commands with realistic Cisco IOS output. This approach is a standard practice in network automation testing — the Python backup script uses identical code to what would run against real Cisco hardware.

Router	SSH Port	IP Address	Configuration Source
Router-HQ	2201	127.0.0.1	Project 7 — HQ router config
Router-Branch	2202	127.0.0.1	Project 7 — Branch router config

Credentials

Parameter	Value
Username	admin
Password	cisco123
Authentication	Password-based SSH

Supported Commands

Command	Response
show running-config	Full device configuration
show version	IOS version and uptime
show ip interface brief	Interface status table
exit	Closes SSH session

4. Backup Script

The backup script (backup_configs.py) is the core automation tool. It iterates through a list of network devices, establishes SSH connections using Paramiko, sends the 'show running-config' command, and saves the output to timestamped files in the backups directory. Each backup file includes metadata headers with the device name, timestamp, and source address.

Script Workflow

Step	Action	Detail
1	Initialize	Load device list, create backup directory
2	Connect	SSH into device using Paramiko
3	Execute	Send 'show running-config' command
4	Save	Write output to timestamped .txt file
5	Log	Append result to backup_log.txt
6	Alert	Send email if any backup fails

Backup File Naming Convention

Each backup file follows the format: **DeviceName_YYYY-MM-DD_HH-MM-SS.txt**. For example: Router-HQ_2026-04-05_17-18-34.txt. This ensures every backup is unique and easily sortable by date.

Key Code Snippet — SSH Connection and Config Pull

```
client = paramiko.SSHClient()
client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
client.connect(hostname=host, port=port,
username=user, password=passwd)

shell = client.invoke_shell()
time.sleep(1)
shell.recv(65535) # Clear welcome banner

shell.send('show running-config\n')
time.sleep(2)
output = shell.recv(65535).decode('utf-8')
```

5. Cron Job Scheduling

A cron job is configured to run the backup automatically every day at 2:00 AM. A wrapper bash script (start_and_backup.sh) handles starting the mock routers, running the backup, and then stopping the mock routers cleanly.

Cron Schedule

Field	Value	Meaning
Minute	0	At minute 0
Hour	2	At 2:00 AM
Day of Month	*	Every day
Month	*	Every month
Day of Week	*	Every day of the week

Cron Entry

```
0 2 * * * /bin/bash /home/admin/network-automation/start_and_backup.sh >>
/home/admin/network-automation/cron_log.txt 2>&1
```

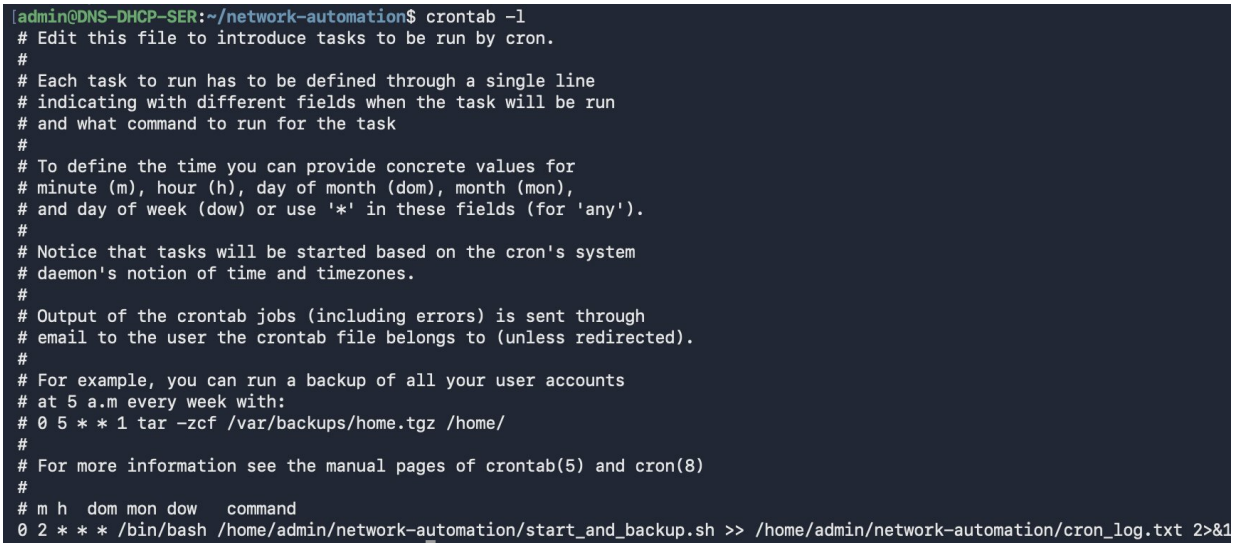

Figure 1: Cron job configured via crontab -l

6. Email Alerting

The backup script includes built-in email alerting functionality. When a backup fails — due to a connection timeout, authentication failure, or any other error — the script automatically sends an email alert with details about which device failed and the error message. Email is sent via SMTP using Python's smtplib module.

Email Configuration

Parameter	Value
SMTP Server	smtp.gmail.com
SMTP Port	587 (TLS)
Authentication	App Password (Gmail)
Trigger	Any device backup failure
Content	Device name, error message, timestamp

Note: Email alerting is disabled by default (EMAIL_ENABLED = False) in the demo environment. To enable it in production, set EMAIL_ENABLED = True and configure valid SMTP credentials.

7. Verification & Testing

Test 1: Backup Script Execution

The backup script was executed via the wrapper script (start_and_backup.sh). Both Router-HQ and Router-Branch were backed up successfully with zero failures.

```
admin@DNS-DHCP-SER:~/network-automation$ bash ~/network-automation/start_and_backup.sh
[*] Starting mock routers...
==================================================
  Mock Cisco Router Simulator
  Router-HQ:     port 2201
  Router-Branch:  port 2202
  Username: admin  Password: cisco123
==================================================
[+] Router-HQ listening on port 2201
[+] Router-Branch listening on port 2202
[*] Running backup...
==================================================
  Network Configuration Backup
  Time: 2026-04-05 17:18:34
==================================================
[*] Connecting to Router-HQ...
[+] Router-HQ: Connection from 127.0.0.1
[*] Pulling config from Router-HQ...
[+] SUCCESS: Router-HQ -> Router-HQ_2026-04-05_17-18-34.txt
[*] Connecting to Router-Branch...
[+] Router-Branch: Connection from 127.0.0.1
[*] Pulling config from Router-Branch...
[+] SUCCESS: Router-Branch -> Router-Branch_2026-04-05_17-18-37.txt

==================================================
  BACKUP SUMMARY
==================================================
  [OK]   Router-HQ -> Router-HQ_2026-04-05_17-18-34.txt
  [OK]   Router-Branch -> Router-Branch_2026-04-05_17-18-37.txt

  Total: 2 | Passed: 2 | Failed: 0
==================================================
[*] Stopping mock routers...
[*] Done!
admin@DNS-DHCP-SER:~/network-automation$
```

Figure 2: Backup script output — Total: 2 | Passed: 2 | Failed: 0

Test 2: Backup Files Created

Multiple timestamped backup files are saved in the backups directory, confirming that repeated runs create new files without overwriting previous backups.

```
admin@DNS-DHCP-SER:~/network-automation$ ls -la ~/network-automation/backups/
total 24
drwxrwxr-x 2 admin admin 4096 Apr  5 17:18 .
drwxrwxr-x 3 admin admin 4096 Apr  5 17:17 ..
-rw-rw-r-- 1 admin admin 1106 Apr  5 17:16 Router-Branch_2026-04-05_17-15-57.txt
-rw-rw-r-- 1 admin admin 1106 Apr  5 17:18 Router-Branch_2026-04-05_17-18-37.txt
-rw-rw-r-- 1 admin admin 1078 Apr  5 17:15 Router-HQ_2026-04-05_17-15-54.txt
-rw-rw-r-- 1 admin admin 1078 Apr  5 17:18 Router-HQ_2026-04-05_17-18-34.txt
admin@DNS-DHCP-SER:~/network-automation$
```

Figure 3: Backup directory listing — timestamped config files for both routers

Test 3: Config File Contents

The saved backup file contains the full running configuration with metadata headers showing the device name, backup timestamp, and source address.

```
admin@DNS-DHCP-SER:~/network-automation$ cat ~/network-automation/backups/Router-HQ_*.txt | head -40
! Backup: Router-HQ
! Date: 2026-04-05_17-15-54
! Source: 127.0.0.1:2201
==============================================



Building configuration...

Current configuration : 2847 bytes
!
version 15.4
service timestamps debug datetime msec
service timestamps log datetime msec
!
hostname Router-HQ
!
boot-start-marker
boot-end-marker
!
enable secret 5 $1$abc$XYZhashvalue
!
no aaa new-model
!
ip cef
!
interface GigabitEthernet0/0
 description LAN-TO-CORE-SW
 ip address 192.168.1.1 255.255.255.0
 duplex auto
 speed auto
!
interface GigabitEthernet0/1
 description WAN-TO-BRANCH
 ip address 10.0.0.1 255.255.255.252
 duplex auto
 speed auto
!
interface Tunnel10
 description GRE-TO-BRANCH
```

Figure 4: Router-HQ backup file — full running config with timestamp header

Test 4: Backup Log

The backup log tracks every execution with pass/fail counts, providing an audit trail for compliance and troubleshooting.

```
admin@DNS-DHCP-SER:~/network-automation$ cat ~/network-automation/backup_log.txt
2026-04-05 17:14:12 | Passed: 0 | Failed: 2
2026-04-05 17:16:00 | Passed: 2 | Failed: 0
2026-04-05 17:18:40 | Passed: 2 | Failed: 0
```

Figure 5: Backup log showing three successful runs

8. Project File Structure

File	Purpose
mock_routers.py	SSH mock simulator — emulates Router-HQ and Router-Branch
backup_configs.py	Main backup script — connects, pulls config, saves with timestamp
start_and_backup.sh	Wrapper script — starts mock routers, runs backup, cleans up
backup_log.txt	Audit log — tracks every backup run with pass/fail counts
backups/	Directory containing all timestamped config backup files
cron_log.txt	Cron output log for scheduled runs

9. Lessons Learned

Paramiko vs Netmiko: While Netmiko provides higher-level abstractions for network devices, Paramiko offers more control over raw SSH sessions. For devices with non-standard prompts or custom SSH implementations, Paramiko's invoke_shell() method provides the flexibility needed.

Mock Testing in Network Automation: Building mock device simulators is a standard industry practice. It allows safe development and testing of automation scripts without risking production network devices. The same Python code works identically against real Cisco hardware.

Timestamped Backups: Using timestamp-based filenames (YYYY-MM-DD_HH-MM-SS) ensures every backup is unique, sortable, and creates a natural version history of device configurations.

Cron Job Automation: Linux cron is a powerful scheduling tool for recurring tasks. Combining a wrapper bash script with Python scripts allows complex workflows to run unattended.

Error Handling and Alerting: Production automation scripts must handle failures gracefully. Email alerting ensures administrators are notified immediately when backups fail, enabling quick response to connectivity or authentication issues.

Logging for Compliance: Maintaining a backup log with timestamps and pass/fail counts provides an audit trail that is valuable for compliance requirements and troubleshooting.

End of Documentation

Daksh Patel — CCNA Certified — github.com/Daksh2601